

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC**



**TIỂU LUẬN HỌC PHẦN
CƠ SỞ DỮ LIỆU WEB VÀ HỆ THỐNG THÔNG TIN**

**ĐỀ TÀI:
QUẢN LÝ WEBSITE BÁN HÀNG**

Họ và tên	Mã số sinh viên
Hoàng Việt Hưng (Leader)	MSV 20002057
Hoàng Văn Dương	MSV 22001247
Phạm Tuấn Anh	MSV 22001233
Nguyễn Tuấn Anh	MSV 22001232
Lê Xuân Lộc	MSV 23001533

Hà Nội, 2025

Lời nói đầu

Trong kỷ nguyên công nghệ 4.0, các hệ thống thông tin và ứng dụng web ngày càng đóng vai trò quan trọng trong hoạt động sản xuất, kinh doanh và quản lý. Đặc biệt, các trang web bán hàng trực tuyến đã trở thành công cụ không thể thiếu, giúp doanh nghiệp tiếp cận khách hàng nhanh chóng, mở rộng thị trường và nâng cao hiệu quả kinh doanh.

Nhận thức được tầm quan trọng và tính thực tiễn đó, trong khuôn khổ môn học **Cơ sở dữ liệu Web**, chúng em đã lựa chọn và thực hiện đề tài: "**Xây dựng trang web bán hàng**". Đề tài tập trung vào việc thiết kế và xây dựng một hệ thống web hoàn chỉnh, bao gồm giao diện người dùng, xử lý nghiệp vụ và đặc biệt là xây dựng cơ sở dữ liệu nhằm quản lý sản phẩm, người dùng, đơn hàng và các thông tin liên quan một cách hiệu quả.

Báo cáo này là kết quả của quá trình tìm hiểu và nghiên cứu nghiêm túc, từ việc phân tích yêu cầu hệ thống, thiết kế cơ sở dữ liệu, xây dựng các chức năng chính của trang web, cho đến việc triển khai và kiểm thử hệ thống trong môi trường thực nghiệm.

Để hoàn thành dự án này, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến các giảng viên hướng dẫn – **thầy Phạm Duy Phương** và **thầy Vũ Tiến Dũng**. Các thầy đã tận tình truyền đạt kiến thức nền tảng, định hướng phương pháp triển khai và giải đáp những vướng mắc chuyên môn trong suốt quá trình học tập. Đồng thời, chúng em cũng xin cảm ơn Khoa Toán – Cơ – Tin học, Trường Đại học Khoa học Tự nhiên đã tạo điều kiện học tập và nghiên cứu thuận lợi để chúng em hoàn thành môn học.

Mặc dù chúng em đã nỗ lực trong quá trình thực hiện, song do thời gian và kinh nghiệm thực tế còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp và nhận xét quý báu từ các Thầy để đề tài được hoàn thiện và phát triển tốt hơn trong thời gian tới.

Chúng em xin chân thành cảm ơn!

Mục lục

Lời nói đầu	1
1 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN TẢNG	5
1.1 Tổng quan	5
1.2 FastAPI và mô hình Router/Dependency Injection	5
1.2.1 Giới thiệu FastAPI	5
1.2.2 Tầng Router và tổ chức endpoint	6
1.2.3 Dependency Injection trong FastAPI	6
1.3 Pydantic DTO và cơ chế Validation Pipeline	6
1.3.1 Vai trò của DTO trong hệ thống	6
1.3.2 Quy trình Validation Pipeline	6
1.3.3 Ý nghĩa thực tiễn	7
1.4 SQLAlchemy ORM và quản lý cơ sở dữ liệu	7
1.4.1 Giới thiệu ORM	7
1.4.2 SQLAlchemy trong dự án	7
1.4.3 Quản lý session và transaction	8
1.5 Hệ quản trị cơ sở dữ liệu MySQL	8
1.5.1 Tổng quan về MySQL	8
1.5.2 Các tính năng hỗ trợ cho hệ thống bán hàng	8
1.6 Bảo mật: JWT, Bcrypt và OAuth2	8
1.6.1 JWT (JSON Web Token)	8
1.6.2 Bcrypt	9
1.6.3 OAuth2	9
1.7 Quản lý cấu hình và biến môi trường	10
1.8 Docker và Containerization	10
1.8.1 Containerization	10
1.8.2 Docker trong dự án	10
1.9 Frontend: React + TypeScript + Vite + MUI + Redux Toolkit + React Query	11
1.9.1 Tổng quan mô hình SPA	11
1.9.2 Các công nghệ và vai trò trong dự án	11
1.9.3 Lợi ích của kiến trúc Frontend:	12
2 THIẾT KẾ CƠ SỞ DỮ LIỆU	13
2.1 Tổng quan về thiết kế cơ sở dữ liệu	13
2.1.1 Giới thiệu bài toán	13
2.1.2 Mục đích và phạm vi lưu trữ	13
2.1.3 Công nghệ sử dụng	13
2.2 Thiết kế mức quan niệm	14
2.2.1 Xác định các thực thể	14
2.2.2 Xác định các mối quan hệ	15

2.2.3	Mô hình thực thể liên kết	15
2.3	Thiết kế mức logic	16
2.3.1	Lược đồ quan hệ	16
2.4	Thiết kế mức logic	17
2.4.1	Lược đồ quan hệ	17
2.4.2	Chuẩn hóa dữ liệu	18
3	Thiết kế mức vật lý (Physical Design)	19
3.1	Bảng: roles	19
3.2	Bảng: users	19
3.3	Bảng: social_accounts	20
3.4	Bảng: tokens	20
3.5	Bảng: categories	20
3.6	Bảng: products	21
3.7	Bảng: product_images	21
3.8	Bảng: coupons	21
3.9	Bảng: coupon_conditions	22
3.10	Bảng: orders	22
3.11	Bảng: order_details	23
3.12	Bảng: comments	23
3.13	Bảng: favorites	23
3.14	Bảng: flyway_schema_history	23
4	THIẾT KẾ HỆ THỐNG VÀ KIẾN TRÚC PHẦN MỀM	25
4.1	Tổng quan kiến trúc nhiều lớp	25
4.2	Tầng Router (Controller Layer)	25
4.3	Tầng DTO - Thiết kế hợp đồng dữ liệu	26
4.4	Tầng Service – Xử lý logic nghiệp vụ	26
4.5	Tầng Repository - truy cập dữ liệu	27
4.6	Tầng ORM Model - ánh xạ dữ liệu	27
4.7	Dependency Injection và quản lý vòng đời Session	27
4.8	Luồng hoạt động của hệ thống	28
4.8.1	Luồng xử lý tổng quát End-to-End	28
4.8.2	Giai đoạn tiếp nhận và xác thực dữ liệu	28
4.8.3	Giai đoạn điều phối và xử lý nghiệp vụ	29
4.8.4	Giai đoạn truy xuất và lưu trữ dữ liệu	29
4.8.5	Giai đoạn phản hồi dữ liệu	29
4.9	Đánh giá kiến trúc và luồng hoạt động	30
5	CƠ CHẾ XÁC THỰC VÀ PHÂN QUYỀN	31
5.1	Tổng quan Authentication và Authorization	31
5.1.1	Authentication (Xác thực - Xác định “User là ai?”)	31

5.1.2	Authorization (Phân quyền - Xác định “User được làm gì?”)	31
5.1.3	JWT Access/Refresh Token: Vòng đời, Luồng cấp mới và Thu hồi	32
5.1.4	Hash mật khẩu bằng Bcrypt	32
5.1.5	OAuth2 Social Login (Google/Facebook)	33
5.1.6	RBAC (Role-Based Access Control)	33
6	Frontend	35
6.1	Tổng quan về Front-end	35
6.1.1	React trong phát triển Front-end	35
6.1.2	Các React Hooks được sử dụng	35
6.1.3	Material UI (MUI)	35
6.1.4	Các thành phần Material UI sử dụng trong dự án	36
6.1.5	Kết luận	36
6.2	Thiết kế các trang xác thực tài khoản	36
6.2.1	Trang đăng nhập (Sign In)	36
6.2.2	Trang đăng ký (Sign Up)	37
6.2.3	Trang quên mật khẩu (Forgot Password)	38
6.2.4	Trang Home	39
6.2.5	Trang kiểm tra email (Check Email)	40
6.2.6	Trang Home	40
6.2.7	Trang Product	41
6.2.8	Trang danh mục sản phẩm	42
6.2.9	Trang Shopping Cart	42
6.2.10	Trang Thanh Toán và Thông Tin Giao Hàng	43
6.3	Triển khai cơ chế kết nối dữ liệu phía Frontend	44
	Tài liệu tham khảo	45

1. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN TẢNG

1.1 Tổng quan

Dự án website bán hàng được phát triển nhằm đáp ứng nhu cầu tìm kiếm sản phẩm, đặt hàng, thanh toán trực tuyến và theo dõi trạng thái đơn hàng của người dùng cuối. Đồng thời, hệ thống cung cấp các công cụ quản lý toàn diện cho phía quản trị viên trong việc điều hành hàng hóa, đơn hàng, khách hàng cùng các nghiệp vụ vận hành khác. Để đảm bảo tính ổn định, khả năng mở rộng và dễ bảo trì, hệ thống được thiết kế theo kiến trúc tách biệt giữa tầng Backend và Frontend, đồng thời áp dụng các công nghệ phổ biến với hệ sinh thái phát triển mạnh mẽ.

Các công nghệ và công cụ chính được sử dụng trong dự án bao gồm:

- **Backend:** FastAPI framework, Pydantic cho Data Transfer Object (DTO), SQLAlchemy ORM
- **Frontend:** React kết hợp TypeScript và Vite, Material UI (MUI), React Router, Redux Toolkit với redux-persist, React Query, Axios
- **Cơ sở dữ liệu:** MySQL
- **Triển khai:** Docker
- **Quản lý mã nguồn:** Git và GitHub

1.2 FastAPI và mô hình Router/Dependency Injection

1.2.1 Giới thiệu FastAPI

FastAPI là một web framework hiện đại được phát triển cho ngôn ngữ Python, được thiết kế tối ưu cho việc xây dựng REST API với hiệu năng cao và trải nghiệm phát triển vượt trội. Framework này tận dụng các tính năng quan trọng sau:

- Type hints của Python để tự động sinh tài liệu API và thực hiện kiểm tra dữ liệu
- ASGI (Asynchronous Server Gateway Interface) để hỗ trợ xử lý bất đồng bộ hiệu quả
- Tích hợp chặt chẽ với Pydantic cho việc validate dữ liệu đầu vào và đầu ra

FastAPI đặc biệt phù hợp cho các hệ thống yêu cầu tốc độ phát triển nhanh với cấu trúc rõ ràng, API tuân thủ chuẩn dữ liệu nghiêm ngặt, đồng thời dễ dàng trong việc kiểm thử và mở rộng.

1.2.2 Tầng Router và tổ chức endpoint

FastAPI hỗ trợ mô hình tách tuyến (routing) thông qua APIRouter, cho phép phân nhóm các endpoint theo nghiệp vụ cụ thể. Cách tiếp cận này mang lại các lợi ích sau:

- Dễ dàng quản lý mã nguồn, tránh tình trạng file nguồn quá lớn và khó bảo trì
- Thuận tiện trong việc phân quyền truy cập theo nhóm endpoint
- Hỗ trợ mở rộng thêm module mới một cách linh hoạt

1.2.3 Dependency Injection trong FastAPI

Dependency Injection (DI) là cơ chế cho phép FastAPI tự động cung cấp các phụ thuộc (dependencies) cần thiết cho mỗi endpoint. Cơ chế này mang lại các lợi ích quan trọng:

- **Tách bạch logic:** Endpoint chỉ đảm nhiệm vai trò điều phối, không phải xử lý toàn bộ logic nghiệp vụ
- **Dễ dàng kiểm thử:** Có thể mock các dependency khi thực hiện unit test
- **Chuẩn hóa luồng xử lý:** Tất cả endpoint sử dụng chung cơ chế truy cập cơ sở dữ liệu, xác thực và ghi log

1.3 Pydantic DTO và cơ chế Validation Pipeline

1.3.1 Vai trò của DTO trong hệ thống

Data Transfer Object (DTO) là mô hình dữ liệu trung gian được sử dụng để:

- Nhận dữ liệu từ phía client (Request DTO)
- Trả dữ liệu về cho client (Response DTO)

Trong dự án, DTO được xây dựng dựa trên Pydantic BaseModel, tạo ra "hợp đồng dữ liệu" (data contract) rõ ràng giữa tầng Frontend và Backend, đảm bảo tính nhất quán trong giao tiếp giữa hai tầng.

1.3.2 Quy trình Validation Pipeline

Validation pipeline trong FastAPI kết hợp với Pydantic diễn ra theo trình tự sau:

1. Router nhận request body và ánh xạ vào DTO dựa trên type hint
2. Pydantic thực hiện parse và validate:
 - Field validation: kiểm tra kiểu dữ liệu, trường bắt buộc/tùy chọn, độ dài tối thiểu/tối đa

- Format validation: ví dụ kiểm tra định dạng email với EmailStr
 - Model validator (cross-field validation) được thực thi sau khi quá trình parse hoàn tất
3. Nếu dữ liệu hợp lệ, Service layer bắt đầu thực thi logic nghiệp vụ
 4. Khi trả kết quả, Response model tiếp tục serialize dữ liệu đầu ra theo schema đã định nghĩa

1.3.3 Ý nghĩa thực tiễn

Cơ chế validation tự động mang lại các lợi ích thiết thực:

- Chặn dữ liệu không hợp lệ ngay tại điểm tiếp nhận, ngăn chặn dữ liệu sai lệch trước khi đưa vào cơ sở dữ liệu
- Đảm bảo thống nhất định dạng dữ liệu trong toàn hệ thống
- Giảm thiểu lỗi runtime do không khớp kiểu dữ liệu
- Dễ dàng bảo trì và cập nhật khi có thay đổi về schema

1.4 SQLAlchemy ORM và quản lý cơ sở dữ liệu

1.4.1 Giới thiệu ORM

Object-Relational Mapping (ORM) là kỹ thuật ánh xạ giữa bảng và bản ghi trong cơ sở dữ liệu quan hệ (SQL) với các đối tượng và class trong code (Python). Thay vì phải viết câu lệnh SQL thủ công cho mọi thao tác, ORM cho phép thao tác với dữ liệu thông qua các đối tượng trong ngôn ngữ lập trình.

1.4.2 SQLAlchemy trong dự án

SQLAlchemy là một trong những ORM phổ biến nhất cho Python, cung cấp các tính năng:

- Model mapping: định nghĩa cấu trúc bảng thông qua class Python
- Session quản lý transaction một cách nhất quán
- Query builder linh hoạt cho các truy vấn phức tạp
- Hỗ trợ migration thông qua tích hợp với Alembic

Trong dự án, SQLAlchemy đóng vai trò quan trọng trong việc:

- Chuẩn hóa các thao tác CRUD (Create, Read, Update, Delete)
- Tách biệt tầng Repository (data access) khỏi tầng Service (business logic)
- Giảm thiểu rủi ro SQL injection nhờ hạn chế sử dụng raw SQL

1.4.3 Quản lý session và transaction

Session là đơn vị làm việc với cơ sở dữ liệu trong SQLAlchemy:

- Mọi thao tác đọc và ghi dữ liệu được thực hiện thông qua session
- Cơ chế commit/rollback đảm bảo tính toàn vẹn dữ liệu
- FastAPI thường inject session theo từng request thông qua dependency `get_db()`

1.5 Hệ quản trị cơ sở dữ liệu MySQL

1.5.1 Tổng quan về MySQL

MySQL là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở, lưu trữ dữ liệu theo mô hình bảng và hỗ trợ truy vấn bằng ngôn ngữ SQL chuẩn. MySQL đặc biệt phù hợp với các ứng dụng thương mại điện tử nhờ tính ổn định cao và khả năng mở rộng linh hoạt.

1.5.2 Các tính năng hỗ trợ cho hệ thống bán hàng

MySQL cung cấp các năng lực quan trọng cho hệ thống:

- **Ràng buộc dữ liệu:** Các constraints như khóa chính, khóa ngoại, unique giúp đảm bảo tính toàn vẹn dữ liệu
- **Quan hệ dữ liệu:** Dễ dàng mô hình hóa các nghiệp vụ phức tạp như quan hệ giữa User–Order–OrderItem–Product
- **Chỉ mục (index):** Tăng tốc độ truy vấn tìm kiếm sản phẩm và lọc đơn hàng
- **Transaction:** Đảm bảo tính nhất quán khi thực hiện các thao tác như tạo đơn hàng, trừ tồn kho và ghi nhận thanh toán
- **Backup và restore:** Hỗ trợ tốt cho việc sao lưu và phục hồi dữ liệu trong môi trường thực tế

1.6 Bảo mật: JWT, Bcrypt và OAuth2

1.6.1 JWT (JSON Web Token)

JWT là cơ chế phát hành token để xác thực người dùng theo mô hình stateless. Quy trình hoạt động:

1. Khi đăng nhập thành công, server cấp access token (có thể kèm theo refresh token)
2. Client gửi token trong mỗi request thông qua header `Authorization: Bearer <token>`

3. Backend xác minh chữ ký token để xác nhận danh tính và quyền hạn

Vai trò của JWT trong dự án:

- Xác thực phiên đăng nhập của người dùng
- Phân quyền dựa trên vai trò (role-based) thông qua payload (ví dụ: role_id)
- Giảm tải việc lưu trữ session phía server

1.6.2 Bcrypt

Bcrypt là thuật toán băm mật khẩu có tích hợp "salt" và chi phí tính toán cao, giúp chống lại các cuộc tấn công:

- Brute-force attack
- Rainbow table attack

Vai trò trong hệ thống:

- Đảm bảo không bao giờ lưu trữ mật khẩu dạng plain text
- Khi đăng ký: mật khẩu được băm trước khi lưu vào cơ sở dữ liệu
- Khi đăng nhập: xác minh mật khẩu thông qua cơ chế so sánh của bcrypt

1.6.3 OAuth2

OAuth2 là chuẩn ủy quyền (authorization) cho phép đăng nhập và ủy quyền thông qua nhà cung cấp bên thứ ba như Google hoặc Facebook. Quy trình trong dự án:

1. Client nhận credential/token từ nhà cung cấp dịch vụ
2. Backend xác minh token và lấy thông tin người dùng (email, id)
3. Hệ thống tự động tạo mới hoặc liên kết với tài khoản nội bộ

Lợi ích của OAuth2:

- Nâng cao trải nghiệm đăng nhập nhanh chóng
- Giảm phụ thuộc vào hệ thống mật khẩu nội bộ
- Có thể kết hợp với xác thực email đã được provider xác minh

1.7 Quản lý cấu hình và biến môi trường

Trong môi trường triển khai thực tế, ứng dụng cần phân tách cấu hình theo từng môi trường (development, staging, production). Environment Variables được sử dụng để lưu trữ các thông tin cấu hình nhạy cảm và khác biệt giữa các môi trường, bao gồm:

- Chuỗi kết nối cơ sở dữ liệu (host, user, password)
- JWT secret key và thời gian hết hạn token
- OAuth client ID và secret
- CORS allowed origins

Lợi ích của việc sử dụng environment variables:

- Tránh hard-code thông tin nhạy cảm trong mã nguồn
- Dễ dàng triển khai trên nhiều môi trường khác nhau
- Hỗ trợ tốt khi chạy container (Docker) và trong quy trình CI/CD

1.8 Docker và Containerization

1.8.1 Containerization

Containerization là phương pháp đóng gói ứng dụng cùng toàn bộ các phụ thuộc (dependencies) thành một "container" có khả năng chạy nhất quán trên mọi môi trường.

1.8.2 Docker trong dự án

Dự án sử dụng Docker để:

- Đóng gói backend, cơ sở dữ liệu và có thể cả frontend
- Đồng bộ môi trường chạy giữa máy phát triển và máy triển khai
- Đơn giản hóa việc setup cho nhóm phát triển thông qua docker-compose

Lợi ích của Docker:

- Giải quyết vấn đề "works on my machine" (chạy được trên máy tôi)
- Dễ dàng quản lý phiên bản môi trường
- Triển khai nhanh chóng và nhất quán trên nhiều môi trường

1.9 Frontend: React + TypeScript + Vite + MUI + Redux Toolkit + React Query

1.9.1 Tổng quan mô hình SPA

Frontend của dự án được xây dựng theo mô hình Single Page Application (SPA), trong đó ứng dụng được tải một lần duy nhất và thực hiện điều hướng nội bộ thông qua router, hạn chế việc tải lại toàn bộ trang. Mô hình SPA đặc biệt phù hợp với hệ thống bán hàng vì giúp tối ưu hóa trải nghiệm người dùng trong các thao tác thường xuyên như tìm kiếm sản phẩm, thêm vào giỏ hàng, đặt hàng và theo dõi trạng thái đơn hàng.

1.9.2 Các công nghệ và vai trò trong dự án

React + TypeScript: React được sử dụng để xây dựng giao diện theo hướng component hóa với khả năng tái sử dụng cao. TypeScript bổ sung cơ chế kiểm tra kiểu (type-checking) tĩnh, giảm thiểu lỗi runtime và giúp chuẩn hóa dữ liệu trao đổi giữa các thành phần (props, state, API response).

Vite: Vite được sử dụng làm công cụ build và development server. Ưu điểm chính của Vite bao gồm tốc độ khởi động nhanh, hot reload hiệu quả và tối ưu hóa quá trình đóng gói (bundle) khi build production.

Material UI (MUI): MUI là thư viện UI cung cấp hệ thống component chuẩn hóa như Button, Form, Modal, Table và Data Grid. Việc sử dụng MUI giúp giao diện đồng nhất, rút ngắn thời gian triển khai và hỗ trợ tốt cho các màn hình quản trị (quản lý sản phẩm, đơn hàng, khách hàng).

React Router: React Router đảm nhiệm chức năng điều hướng giữa các trang như Login, Home và các trang chức năng khác. Framework này hỗ trợ xây dựng cơ chế route guard (điều hướng theo trạng thái đăng nhập), giúp kiểm soát quyền truy cập vào các trang yêu cầu xác thực.

Redux Toolkit + redux-persist: Redux Toolkit quản lý global state của ứng dụng, bao gồm trạng thái đăng nhập, thông tin người dùng và cấu hình giao diện. Thư viện redux-persist hỗ trợ lưu trữ các state quan trọng xuống localStorage, giúp duy trì trạng thái qua các thao tác refresh hoặc đóng/mở trình duyệt.

React Query + Axios: Axios được sử dụng để thực hiện các HTTP request đến backend. React Query quản lý server state (dữ liệu từ API) với các cơ chế như cache, retry, refetch, đồng bộ dữ liệu và quản lý trạng thái loading/error theo chuẩn.

Notistack: Notistack cung cấp cơ chế thông báo dạng snackbar/toast, được sử dụng để phản hồi nhanh chóng cho người dùng trong các thao tác như đăng nhập, thêm sản phẩm, tạo đơn hàng hoặc thông báo lỗi hệ thống.

1.9.3 Lợi ích của kiến trúc Frontend:

Stack công nghệ React + Redux + React Query giúp frontend đạt được các mục tiêu kỹ thuật quan trọng:

- **Tách bạch UI state và server state:** Redux quản lý trạng thái giao diện và trạng thái toàn cục, trong khi React Query quản lý dữ liệu từ API và đồng bộ hóa theo cache
- **Dễ dàng mở rộng tính năng quản trị:** MUI và Data Grid hỗ trợ mạnh mẽ cho các màn hình dạng bảng (lọc, phân trang, sắp xếp), phù hợp với nghiệp vụ quản lý sản phẩm, đơn hàng và khách hàng
- **Tối ưu hóa trải nghiệm người dùng:** Kết hợp lazy loading, loading states và cơ chế cache giúp giảm độ trễ cảm nhận, tăng tốc độ điều hướng và phản hồi thao tác

2. THIẾT KẾ CƠ SỞ DỮ LIỆU

2.1 Tổng quan về thiết kế cơ sở dữ liệu

2.1.1 Giới thiệu bài toán

Dự án này được xây dựng với mục tiêu cung cấp một nền tảng thương mại điện tử hiện đại, phục vụ nhu cầu giao dịch trực tuyến giữa người mua và người bán. Với đặc thù của hệ thống E-commerce yêu cầu xử lý đồng thời lượng lớn giao dịch và truy xuất dữ liệu liên tục, việc thiết kế cơ sở dữ liệu đóng vai trò then chốt. Hệ thống cần đảm bảo tính nhất quán (Consistency), khả năng sẵn sàng (Availability) và độ bảo mật cao cho dữ liệu nhạy cảm của người dùng.

2.1.2 Mục đích và phạm vi lưu trữ

Cơ sở dữ liệu được thiết kế nhằm phục vụ toàn diện các quy trình nghiệp vụ từ quản trị hệ thống đến trải nghiệm người dùng cuối. Dựa trên phân tích yêu cầu, phạm vi lưu trữ dữ liệu bao gồm các nhóm chức năng chính sau:

- **Quản lý định danh và truy cập (IAM):** Lưu trữ thông tin người dùng (*users*), phân quyền quản trị (*roles*). Đặc biệt, hệ thống hỗ trợ lưu trữ thông tin xác thực đa nền tảng thông qua mạng xã hội (*social_accounts*) và quản lý phiên đăng nhập an toàn thông qua cơ chế token (*tokens*).
- **Quản lý danh mục và sản phẩm:** Tổ chức dữ liệu hàng hóa theo cấu trúc phân cấp (*categories*), lưu trữ chi tiết thông tin sản phẩm (*products*) và thư viện hình ảnh đa phương tiện (*product_images*).
- **Quản lý giao dịch và đơn hàng:** Lưu trữ toàn bộ vòng đời của đơn hàng (*orders*) từ lúc khởi tạo đến khi hoàn tất, bao gồm chi tiết từng mặt hàng (*order_details*), thông tin vận chuyển và thanh toán.
- **Marketing và Khuyến mãi:** Quản lý các chương trình giảm giá thông qua hệ thống mã coupon (*coupons*) và các điều kiện áp dụng phức tạp (*coupon_conditions*).
- **Tương tác người dùng:** Ghi nhận các hoạt động tương tác nhằm nâng cao trải nghiệm mua sắm như: danh sách sản phẩm yêu thích (*favorites*) và bình luận đánh giá sản phẩm (*comments*).

2.1.3 Công nghệ sử dụng

Để đáp ứng các yêu cầu phi chức năng về hiệu năng và khả năng mở rộng, nhóm phát triển quyết định sử dụng Hệ quản trị cơ sở dữ liệu **MySQL phiên bản 8.0**.

- **Lý do lựa chọn:**

- *Tính ổn định và phổ biến:* MySQL là RDBMS mã nguồn mở phổ biến nhất thế giới, có cộng đồng hỗ trợ lớn và tương thích tốt với hầu hết các ngôn ngữ lập trình Backend.
- **ACID Compliance:** Đảm bảo tính toàn vẹn dữ liệu tuyệt đối cho các giao dịch tài chính/đặt hàng – yếu tố sống còn của hệ thống thương mại điện tử.
- **Hiệu năng cao:** Phiên bản 8.0 mang lại cải tiến vượt trội về tốc độ thực thi truy vấn và tối ưu hóa tài nguyên so với các phiên bản cũ.
- **Khả năng mở rộng:** Hỗ trợ tốt cho việc nhân rộng (Replication) và phân mảnh dữ liệu (Sharding) khi hệ thống phát triển lớn trong tương lai.

2.2 Thiết kế mức quan niệm

2.2.1 Xác định các thực thể

Dựa trên quá trình phân tích yêu cầu nghiệp vụ thực tế, hệ thống ShopApp được mô hình hóa với các thực thể chính sau:

- 1. User (Người dùng):** Đại diện cho các tài khoản trong hệ thống. Bao gồm thông tin cá nhân, thông tin xác thực và vai trò (Role) để phân quyền.
- 2. Category (Danh mục):** Phân loại sản phẩm để tổ chức dữ liệu khoa học, hỗ trợ việc lọc và tìm kiếm.
- 3. Product (Sản phẩm):** Lưu trữ thông tin cốt lõi của hàng hóa như tên, giá, mô tả.
- 4. Product Image (Hình ảnh sản phẩm):** Do một sản phẩm thực tế cần nhiều góc chụp, thực thể này được tách riêng để lưu trữ danh sách hình ảnh chi tiết cho từng sản phẩm.
- 5. Order (Đơn hàng):** Đại diện cho giao dịch mua hàng, lưu trữ tổng tiền, trạng thái vận chuyển, thanh toán và thông tin người nhận.
- 6. Order Detail (Chi tiết đơn hàng):** Thực thể trung gian giải quyết quan hệ nhiều-nhiều giữa Đơn hàng và Sản phẩm, ghi nhận chính xác giá bán và số lượng tại thời điểm giao dịch.
- 7. Coupon (Mã giảm giá):** Lưu trữ thông tin về các chương trình khuyến mãi, điều kiện áp dụng và mức giảm giá.
- 8. Interaction (Tương tác):** Bao gồm các thực thể **Comment** (Bình luận) và **Favorite** (Yêu thích), ghi lại phản hồi và sự quan tâm của người dùng đối với sản phẩm.
- 9. Authentication Data:** Bao gồm **Token** (quản lý phiên đăng nhập) và **Social Account** (liên kết đăng nhập qua Google/Facebook).

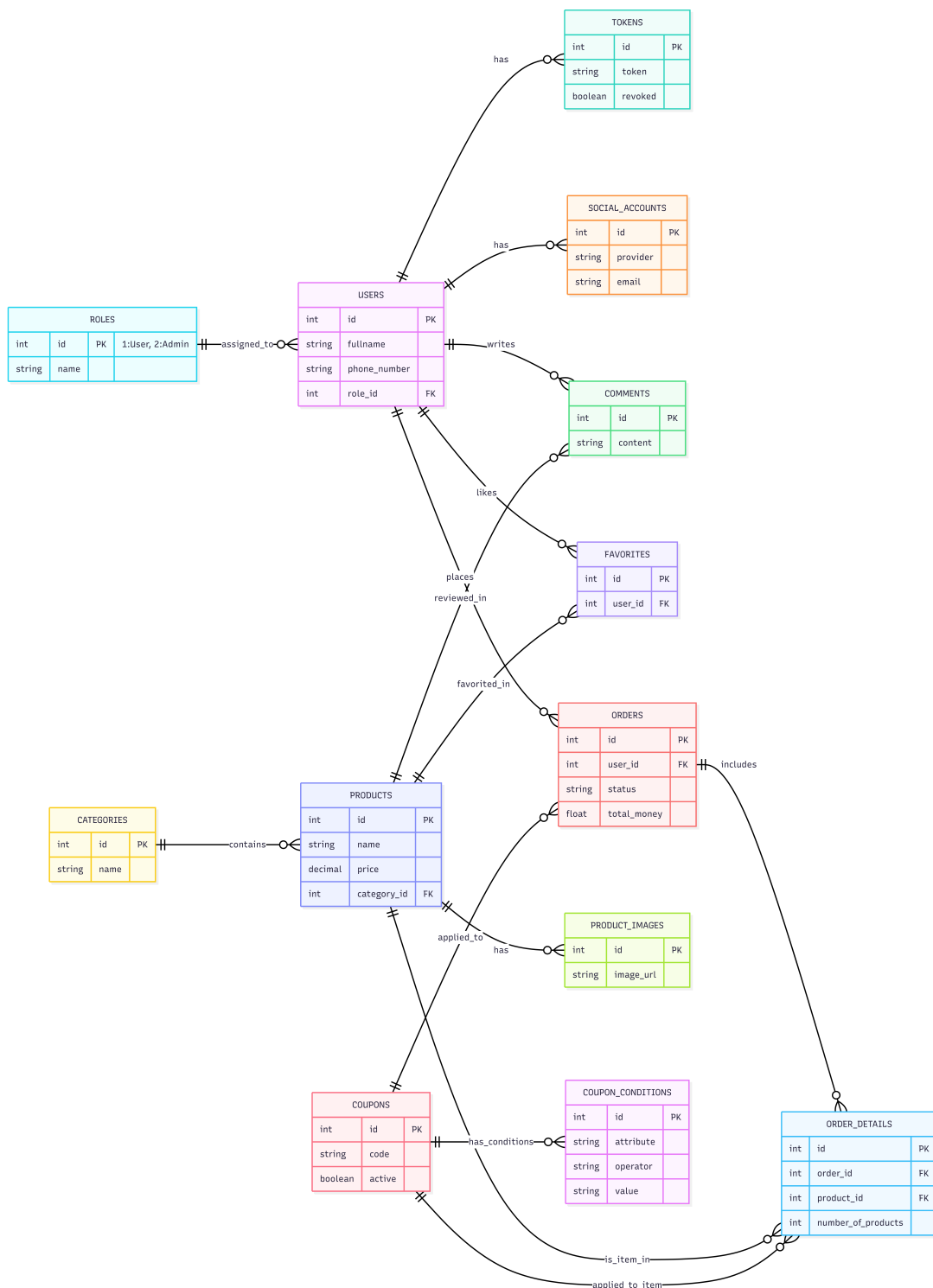
2.2.2 Xác định các mối quan hệ

Các quy tắc nghiệp vụ quy định mối liên kết giữa các dữ liệu như sau:

- **Category – Product (1-n):** Một danh mục chứa nhiều sản phẩm, một sản phẩm thuộc về một danh mục.
- **Product – Product Image (1-n):** Một sản phẩm có thể có nhiều hình ảnh minh họa đi kèm.
- **User – Order (1-n):** Một người dùng có thể thực hiện nhiều đơn hàng, nhưng mỗi đơn hàng thuộc về một tài khoản duy nhất.
- **Order – Product (n-n):** Quan hệ nhiều-nhiều được hiện thực hóa thông qua thực thể *Order Detail*.
- **Order – Coupon (n-1):** Một đơn hàng có thể áp dụng một mã giảm giá (hoặc không), một mã giảm giá có thể áp dụng cho nhiều đơn hàng (tùy theo số lượng phát hành).
- **User – Product (Tương tác):** Người dùng có thể bình luận hoặc thêm nhiều sản phẩm vào danh sách yêu thích.

2.2.3 Mô hình thực thể liên kết

Sơ đồ ERD dưới đây mô tả trực quan cấu trúc các thực thể và mối quan hệ trong cơ sở dữ liệu ShopApp:



Hình 1: Sơ đồ quan hệ thực thể (ERD) của hệ thống ShopApp

2.3 Thiết kế mức logic

2.3.1 Lược đồ quan hệ

Từ mô hình ERD, chúng tôi chuyển đổi các thực thể và mối quan hệ thành các lược đồ quan hệ cụ thể.

Quy ước: Gạch chân là Khóa chính (Primary Key - PK), In nghiêng là Khóa ngoại (Foreign Key - FK).

- **ROLES**(id, name)
- **USERS**(id, fullname, phone_number, address, password, created_at, updated_at, is_active, date_of_birth, facebook_account_id, google_account_id, *role_id*)
- **TOKENS**(id, token, token_type, expiration_date, revoked, expired, *user_id*)
- **CATEGORIES**(id, name)
- **PRODUCTS**(id, name, price, thumbnail, description, created_at, updated_at, *category_id*)
- **PRODUCT_IMAGES**(id, image_url, *product_id*)
- **ORDERS**(id, fullname, email, phone_number, address, note, order_date, status, total_money, shipping_method, shipping_address, shipping_date, payment_method, active, *user_id*)
- **ORDER_DETAILS**(id, price, number_of_products, total_money, color, *order_id*, *product_id*)

2.4 Thiết kế mức logic

2.4.1 Lược đồ quan hệ

Từ mô hình ERD, chúng tôi chuyển đổi các thực thể và mối quan hệ thành các lược đồ quan hệ cụ thể.

Quy ước: Gạch chân là Khóa chính - PK, In nghiêng là Khóa ngoại - FK.

- **ROLES**(id, name)
- **USERS**(id, fullname, phone_number, address, password, email, date_of_birth, is_active, facebook_account_id, google_account_id, *role_id*)
- **SOCIAL_ACCOUNTS**(id, provider, provider_id, email, name, *user_id*)
- **TOKENS**(id, token, token_type, expiration_date, revoked, expired, *user_id*)
- **CATEGORIES**(id, name)
- **PRODUCTS**(id, name, price, thumbnail, description, created_at, updated_at, *category_id*)
- **PRODUCT_IMAGES**(id, image_url, *product_id*)
- **COUPONS**(id, code, active)
- **COUPON_CONDITIONS**(id, attribute, operator, value, discount_amount, *coupon_id*)
- **ORDERS**(id, fullname, email, phone_number, address, note, order_date, status, total_money, shipping_method, shipping_address, shipping_date, payment_method, active, *user_id*, *coupon_id*)

- **ORDER_DETAILS**(id, price, number_of_products, total_money, color, *order_id*, *product_id*, *coupon_id*)
- **COMMENTS**(id, content, created_at, updated_at, *product_id*, *user_id*)
- **FAVORITES**(id, *user_id*, *product_id*)

2.4.2 Chuẩn hóa dữ liệu

Quy trình chuẩn hóa được áp dụng để loại bỏ dư thừa dữ liệu và đảm bảo tính nhất quán.

Dạng chuẩn 1 - 1NF

- **Yêu cầu:** Các giá trị trong mỗi cột phải là nguyên tử và không chứa nhóm lặp.
- **Áp dụng:**
 - Trong bảng **USERS**, địa chỉ không lưu dạng danh sách nhiều nơi ở mà chỉ lưu một địa chỉ giao hàng mặc định.
 - Một sản phẩm có nhiều ảnh: Chúng tôi không lưu chuỗi URL ảnh cách nhau bởi dấu phẩy trong bảng **PRODUCTS**. Thay vào đó, chúng tôi tách ra bảng riêng **PRODUCT_IMAGES**, đảm bảo mỗi dòng chỉ chứa 1 URL ảnh duy nhất.

Dạng chuẩn 2 - 2NF

- **Yêu cầu:** Đạt 1NF và các thuộc tính không khóa phải phụ thuộc hoàn toàn vào khóa chính.
- **Áp dụng:** Trong bảng **ORDER_DETAILS**, thay vì dùng khóa chính phức hợp gồm *order_id*, *product_id*, chúng tôi sử dụng khóa chính giả là cột *id* tự tăng. Điều này giúp việc tham chiếu và quản lý dữ liệu đơn giản hơn trên các công cụ ORM, đồng thời đảm bảo mọi thuộc tính như giá bán, số lượng đều phụ thuộc hoàn toàn vào *id*.

Dạng chuẩn 3 - 3NF và Phi chuẩn hóa

- **Tuân thủ 3NF:** Trong bảng **PRODUCTS**, chúng tôi chỉ lưu *category_id* thay vì lưu cả tên danh mục. Tên danh mục được tách ra bảng **CATEGORIES**. Điều này loại bỏ phụ thuộc bắc cầu, giúp việc cập nhật tên danh mục chỉ cần thực hiện tại một nơi duy nhất.
- **Ngoại lệ:** Mặc dù lý thuyết 3NF yêu cầu không lưu trữ các thuộc tính có thể tính toán được, nhưng trong bảng **ORDERS**, chúng tôi vẫn lưu cột *total_money*.
Giải trình: Đây là kỹ thuật Phi chuẩn hóa có chủ đích. Trong các hệ thống thương mại điện tử, tần suất xem lịch sử đơn hàng rất lớn. Việc lưu sẵn tổng tiền giúp truy vấn nhanh chóng mà không cần thực hiện phép tính tổng từ bảng chi tiết mỗi lần người dùng xem đơn hàng, giúp tối ưu hóa hiệu năng hệ thống.

3. Thiết kế mức vật lý (Physical Design)

Hệ thống bao gồm 14 bảng dữ liệu, chi tiết như sau:

3.1 Bảng: roles

Field	Type	Constraint	Description
id	INT	PK	Mã quyền
name	VARCHAR(20)	NOT NULL	Tên quyền (admin, user)

Bảng 1: Từ điển dữ liệu bảng Roles

3.2 Bảng: users

Field	Type	Constraint	Description
id	INT	PK, Auto Increment	ID người dùng
fullname	VARCHAR(100)	DEFAULT ''	Tên đầy đủ
phone_number	VARCHAR(15)	NULL	Số điện thoại
address	VARCHAR(200)	DEFAULT ''	Địa chỉ
password	CHAR(60)	NOT NULL	Mật khẩu mã hóa
email	VARCHAR(255)	DEFAULT ''	Email
date_of_birth	DATE	NULL	Ngày sinh
is_active	TINYINT(1)	DEFAULT 1	Trạng thái kích hoạt
facebook_account_id	VARCHAR(100)	DEFAULT NULL	ID Facebook (nếu có)
google_account_id	VARCHAR(100)	DEFAULT NULL	ID Google (nếu có)
profile_image	VARCHAR(255)	DEFAULT ''	Ảnh đại diện
role_id	INT	FK (roles)	Vai trò người dùng
created_at	DATETIME	NULL	Ngày tạo
updated_at	DATETIME	NULL	Ngày cập nhật

Bảng 2: Từ điển dữ liệu bảng Users

3.3 Bảng: social_accounts

Field	Type	Constraint	Description
id	INT	PK, Auto Increment	ID
provider	VARCHAR(20)	NOT NULL	Nhà cung cấp (Google, FB)
provider_id	VARCHAR(50)	NOT NULL	ID từ nhà cung cấp
email	VARCHAR(150)	NOT NULL	Email tài khoản xã hội
name	VARCHAR(100)	NOT NULL	Tên người dùng
user_id	INT	FK (users)	Liên kết tài khoản hệ thống

Bảng 3: Từ điển dữ liệu bảng Social Accounts

3.4 Bảng: tokens

Field	Type	Constraint	Description
id	INT	PK, Auto Increment	ID
token	VARCHAR(255)	UNIQUE, NOT NULL	Access Token
token_type	VARCHAR(50)	NOT NULL	Loại token
expiration_date	DATETIME	NULL	Ngày hết hạn
revoked	TINYINT(1)	NOT NULL	Đã thu hồi?
expired	TINYINT(1)	NOT NULL	Đã hết hạn?
is_mobile	TINYINT(1)	DEFAULT 0	Thiết bị mobile?
refresh_token	VARCHAR(255)	DEFAULT ''	Refresh Token
refresh_expiration_date	DATETIME	NULL	Hạn Refresh Token
user_id	INT	FK (users)	Người dùng sở hữu

Bảng 4: Từ điển dữ liệu bảng Tokens

3.5 Bảng: categories

Field	Type	Constraint	Description
id	INT	PK, Auto Increment	ID danh mục
name	VARCHAR(50)	UNIQUE	Tên danh mục

Bảng 5: Từ điển dữ liệu bảng Categories

3.6 Bảng: products

Field	Type	Constraint	Description
id	INT	PK, Auto Increment	ID sản phẩm
name	VARCHAR(350)	NULL	Tên sản phẩm
price	DECIMAL(10,2)	NULL	Giá bán
thumbnail	VARCHAR(255)	NULL	Ảnh thu nhỏ
description	LONGTEXT	NULL	Mô tả chi tiết
category_id	INT	FK (categories)	Danh mục
created_at	DATETIME	NULL	Ngày tạo
updated_at	DATETIME	NULL	Ngày cập nhật

Bảng 6: Từ điển dữ liệu bảng Products

3.7 Bảng: product_images

Field	Type	Constraint	Description
id	INT	PK, Auto Increment	ID
image_url	VARCHAR(300)	NULL	Đường dẫn ảnh
product_id	INT	FK (products)	Sản phẩm liên quan

Bảng 7: Từ điển dữ liệu bảng Product Images

3.8 Bảng: coupons

Field	Type	Constraint	Description
id	INT	PK, Auto Increment	ID
code	VARCHAR(50)	NOT NULL	Mã code giảm giá
active	TINYINT(1)	DEFAULT 1	Trạng thái hoạt động

Bảng 8: Từ điển dữ liệu bảng Coupons

3.9 Bảng: coupon_conditions

Field	Type	Constraint	Description
id	INT	PK, Auto Increment	ID
attribute	VARCHAR(255)	NOT NULL	Thuộc tính kiểm tra
operator	VARCHAR(10)	NOT NULL	Toán tử so sánh
value	VARCHAR(255)	NOT NULL	Giá trị so sánh
discount_amount	DECIMAL(5,2)	NOT NULL	Số tiền/phần trăm giảm
coupon_id	INT	FK (coupons)	Thuộc mã giảm giá nào

Bảng 9: Từ điển dữ liệu bảng Coupon Conditions

3.10 Bảng: orders

Field	Type	Constraint	Description
id	INT	PK, Auto Increment	ID đơn hàng
fullname	VARCHAR(100)	DEFAULT ''	Tên người nhận
email	VARCHAR(100)	DEFAULT ''	Email người nhận
phone_number	VARCHAR(20)	NOT NULL	SĐT người nhận
address	VARCHAR(200)	NOT NULL	Địa chỉ giao hàng
note	VARCHAR(100)	DEFAULT ''	Ghi chú
order_date	DATETIME	CURRENT_TIMESTAMP	Ngày đặt
status	ENUM	(...)	Trạng thái đơn hàng
total_money	FLOAT	NULL	Tổng tiền
shipping_method	VARCHAR(100)	NULL	Phương thức vận chuyển
shipping_address	VARCHAR(200)	NULL	Địa chỉ ship thực tế
shipping_date	DATE	NULL	Ngày giao
tracking_number	VARCHAR(100)	NULL	Mã vận đơn
payment_method	VARCHAR(100)	NULL	Phương thức thanh toán
active	TINYINT(1)	NULL	Hiệu lực
user_id	INT	FK (users)	Người đặt
coupon_id	INT	FK (coupons)	Mã giảm giá áp dụng

Bảng 10: Từ điển dữ liệu bảng Orders

3.11 Bảng: order_details

Field	Type	Constraint	Description
id	INT	PK, Auto Increment	ID
price	DECIMAL(10,2)	NULL	Giá chốt tại thời điểm mua
number_of_products	INT	DEFAULT 1	Số lượng
total_money	DECIMAL(10,2)	DEFAULT 0.00	Thành tiền
color	VARCHAR(20)	DEFAULT ''	Màu sắc
order_id	INT	FK (orders)	Thuộc đơn hàng
product_id	INT	FK (products)	Sản phẩm
coupon_id	INT	FK (coupons)	Coupon áp dụng (nếu có)

Bảng 11: Từ điển dữ liệu bảng Order Details

3.12 Bảng: comments

Field	Type	Constraint	Description
id	INT	PK, Auto Increment	ID
content	VARCHAR(255)	NULL	Nội dung bình luận
product_id	INT	FK (products)	Sản phẩm được bình luận
user_id	INT	FK (users)	Người bình luận
created_at	DATETIME	NULL	Thời gian tạo
updated_at	DATETIME	NULL	Thời gian sửa

Bảng 12: Từ điển dữ liệu bảng Comments

3.13 Bảng: favorites

Field	Type	Constraint	Description
id	INT	PK, Auto Increment	ID
user_id	INT	FK (users)	Người dùng
product_id	INT	FK (products)	Sản phẩm yêu thích

Bảng 13: Từ điển dữ liệu bảng Favorites

3.14 Bảng: flyway_schema_history

Lưu ý: Đây là bảng kỹ thuật được tạo tự động bởi công cụ Flyway để quản lý lịch sử migration database.

Field	Type	Constraint	Description
installed_rank	INT	PK	Thứ tự cài đặt
version	VARCHAR(50)	NULL	Phiên bản
description	VARCHAR(200)	NOT NULL	Mô tả migration
type	VARCHAR(20)	NOT NULL	Loại script (SQL, JDBC...)
script	VARCHAR(1000)	NOT NULL	Tên file script
checksum	INT	NULL	Mã kiểm tra thay đổi
installed_on	TIMESTAMP	DEFAULT NOW	Thời điểm cài đặt
success	TINYINT(1)	NOT NULL	Trạng thái thành công

Bảng 14: Bảng quản lý lịch sử Migration (Flyway)

4. THIẾT KẾ HỆ THỐNG VÀ KIẾN TRÚC PHẦN MỀM

4.1 Tổng quan kiến trúc nhiều lớp

Hệ thống backend của dự án được thiết kế dựa trên mô hình kiến trúc nhiều lớp (Layered Architecture), nhằm mục tiêu tách bạch rõ ràng các trách nhiệm giữa các thành phần, giảm thiểu sự phụ thuộc chéo và nâng cao khả năng mở rộng, kiểm thử cũng như bảo trì hệ thống.

Kiến trúc được tổ chức thành các lớp chính như sau:

- **Router (Controller Layer):** Tầng điều khiển tiếp nhận và định tuyến các yêu cầu
- **DTO (Data Transfer Object):** Đối tượng truyền tải dữ liệu
- **Service (Business Logic Layer):** Tầng xử lý logic nghiệp vụ
- **Repository (Data Access Layer):** Tầng truy cập dữ liệu
- **Model (ORM Layer):** Tầng ánh xạ quan hệ đối tượng
- **Response Model:** Mô hình dữ liệu phản hồi

Mỗi lớp trong kiến trúc đảm nhiệm một vai trò duy nhất, tuân thủ nguyên tắc Single Responsibility Principle (SRP) trong Clean Architecture, đảm bảo tính mô-đun hóa và khả năng tái sử dụng cao.

4.2 Tầng Router (Controller Layer)

Tầng Router đóng vai trò là điểm tiếp nhận các yêu cầu từ phía client, được triển khai thông qua APIRouter của FastAPI. Router không chứa logic nghiệp vụ mà chỉ thực hiện các nhiệm vụ sau:

- Định nghĩa các endpoint và phương thức HTTP tương ứng
- Ánh xạ request body vào các đối tượng DTO
- Inject các service cần thiết thông qua cơ chế Dependency Injection
- Xử lý lỗi ở cấp độ HTTP (status code, response format)

Tầng Router tuân thủ nguyên tắc không thao tác trực tiếp với cơ sở dữ liệu, không thực hiện các tác vụ như hash mật khẩu hay sinh token, đảm bảo ranh giới rõ ràng giữa điều phối (orchestration) và xử lý nghiệp vụ (business logic).

4.3 Tầng DTO - Thiết kế hợp đồng dữ liệu

Data Transfer Object (DTO) được xây dựng dựa trên Pydantic BaseModel, đóng vai trò là "hợp đồng dữ liệu" (data contract) giữa tầng frontend và backend.

Các vai trò chính của DTO bao gồm:

- **Validate dữ liệu đầu vào:** Kiểm tra kiểu dữ liệu, định dạng và các ràng buộc
- **Chuẩn hóa dữ liệu đầu ra:** Đảm bảo tính nhất quán trong cấu trúc phản hồi
- **Cách ly internal model:** Tách biệt mô hình ORM nội bộ khỏi client, bảo vệ cấu trúc dữ liệu bên trong

DTO thể hiện validation pipeline ngay tại tầng API, đảm bảo dữ liệu không hợp lệ được loại bỏ trước khi đi sâu vào các tầng xử lý bên trong của hệ thống.

4.4 Tầng Service – Xử lý logic nghiệp vụ

Tầng Service là nơi tập trung toàn bộ logic nghiệp vụ của hệ thống. Các chức năng điển hình bao gồm:

- Xử lý đăng ký và đăng nhập người dùng
- Hash và xác minh mật khẩu
- Sinh JWT và refresh token
- Xử lý đăng nhập thông qua bên thứ ba (Google, Facebook)
- Kiểm tra các điều kiện nghiệp vụ

Trong kiến trúc này, Router chỉ đóng vai trò gọi Service, trong khi Service chịu trách nhiệm:

- Gọi các phương thức từ Repository
- Điều phối nhiều Repository khi cần thiết
- Áp dụng các quy tắc nghiệp vụ (business rules)

Cách tiếp cận này giúp logic nghiệp vụ không phụ thuộc vào FastAPI, tăng khả năng tái sử dụng và dễ dàng thực hiện unit testing.

4.5 Tầng Repository - truy cập dữ liệu

Repository là tầng duy nhất được phép truy cập trực tiếp vào cơ sở dữ liệu. Mọi thao tác CRUD (Create, Read, Update, Delete) đều phải đi qua tầng này.

Trách nhiệm của Repository bao gồm:

- Thực hiện các truy vấn dữ liệu thông qua ORM
- Không chứa business logic
- Độc lập với các thành phần HTTP, DTO và JWT

Lợi ích của Repository Pattern:

- **Tính linh hoạt:** Việc thay đổi cơ sở dữ liệu (ví dụ từ MySQL sang PostgreSQL) ít ảnh hưởng đến tầng Service
- **Khả năng kiểm thử:** Có thể mock Repository trong quá trình testing
- **Giảm coupling:** Giảm sự phụ thuộc giữa business logic và database engine

4.6 Tầng ORM Model - ánh xạ dữ liệu

Các Model được xây dựng bằng SQLAlchemy ORM, đại diện cho các bảng trong cơ sở dữ liệu.

Trách nhiệm của tầng Model:

- Ánh xạ giữa bảng và cột trong database
- Định nghĩa các quan hệ (relationships) giữa các entity
- Không thực hiện validate dữ liệu request hay xử lý logic nghiệp vụ

Việc tách biệt rõ ràng giữa Model và DTO giúp bảo vệ cấu trúc database khỏi sự thay đổi từ phía client và ngược lại.

4.7 Dependency Injection và quản lý vòng đời Session

FastAPI sử dụng cơ chế Dependency Injection thông qua hàm `Depends()` để thực hiện các nhiệm vụ:

- Inject DB Session cho mỗi request
- Inject các Service cần thiết
- Inject thông tin người dùng hiện tại (từ JWT)

Lợi ích của cơ chế này:

- DB session được quản lý theo vòng đời của từng request
- Tự động đóng session sau khi xử lý hoàn tất, tránh memory leak
- Dễ dàng mock các dependency trong quá trình kiểm thử

4.8 Luồng hoạt động của hệ thống

Luồng hoạt động của hệ thống mô tả cách các thành phần trong kiến trúc backend phối hợp với nhau trong quá trình xử lý một yêu cầu từ phía client, từ thời điểm tiếp nhận request cho đến khi trả về response. Phần này tập trung vào hành vi vận hành (runtime behavior) của hệ thống, dựa trên kiến trúc nhiều lớp đã được trình bày ở các mục trước.

Việc mô tả rõ ràng luồng hoạt động giúp làm sáng tỏ cách hệ thống đảm bảo tính toàn vẹn dữ liệu, kiểm soát nghiệp vụ và bảo mật trong suốt vòng đời của một request.

4.8.1 Luồng xử lý tổng quát End-to-End

Một request từ phía client khi đi vào hệ thống backend sẽ được xử lý theo chuỗi các bước sau:

Request → DTO Validation → Router → Service → Repository → Database

Database → Repository → Service → Response DTO → Client

Luồng xử lý này đảm bảo mỗi tầng trong kiến trúc chỉ thực hiện đúng vai trò của mình, tuân thủ nguyên tắc phân tách trách nhiệm (Separation of Concerns) và giảm thiểu sự phụ thuộc chéo giữa các thành phần.

4.8.2 Giai đoạn tiếp nhận và xác thực dữ liệu

Khi người dùng thực hiện một thao tác trên giao diện frontend (ví dụ: đăng ký tài khoản, đăng nhập, đặt hàng), trình duyệt sẽ gửi một HTTP request chứa dữ liệu ở định dạng JSON đến hệ thống backend.

Ngay tại tầng API, dữ liệu request được ánh xạ vào các đối tượng DTO (Data Transfer Object) dựa trên Pydantic. Tại đây, hệ thống thực hiện validation pipeline, bao gồm:

- Kiểm tra kiểu dữ liệu (type checking)
- Kiểm tra các ràng buộc (constraints) như độ dài mật khẩu, định dạng email
- Kiểm tra logic liên trường (ví dụ: bắt buộc phải có email hoặc số điện thoại)

Nếu dữ liệu không hợp lệ, hệ thống dừng luồng xử lý và trả về thông báo lỗi tương ứng mà không cho phép request đi sâu vào các tầng xử lý bên trong. Cơ chế này giúp bảo vệ tầng Service và Repository khỏi dữ liệu không tin cậy, đồng thời giảm thiểu nguy cơ lỗi nghiệp vụ.

4.8.3 Giai đoạn điều phối và xử lý nghiệp vụ

Sau khi dữ liệu được xác thực thành công, Router tiếp nhận request và sử dụng cơ chế Dependency Injection của FastAPI để inject các Service cần thiết.

Tầng Service đóng vai trò trung tâm trong luồng xử lý, chịu trách nhiệm thực thi toàn bộ logic nghiệp vụ của hệ thống, bao gồm:

- Xử lý đăng ký và đăng nhập người dùng
- Hash và xác minh mật khẩu thông qua Bcrypt
- Sinh access token và refresh token theo chuẩn JWT
- Kiểm tra quyền truy cập của người dùng dựa trên thông tin xác thực
- Điều phối nhiều Repository khi một nghiệp vụ yêu cầu truy xuất nhiều bảng dữ liệu

Trong các request yêu cầu xác thực, các dependency như `get_current_user` sẽ được gọi để giải mã JWT, xác định danh tính và vai trò (User/Admin) của người dùng trước khi cho phép tiếp tục xử lý.

4.8.4 Giai đoạn truy xuất và lưu trữ dữ liệu

Tầng Service không trực tiếp thao tác với cơ sở dữ liệu mà gọi các phương thức tương ứng từ tầng Repository. Repository là tầng duy nhất chịu trách nhiệm giao tiếp với hệ quản trị cơ sở dữ liệu thông qua SQLAlchemy ORM.

Trong giai đoạn này:

- Các truy vấn được xây dựng dưới dạng ORM query
- Các phép join giữa nhiều bảng (ví dụ: `product - order_details`) được thực hiện tại Repository
- DB Session được quản lý theo cơ chế Unit of Work, đảm bảo các giao dịch tuân thủ tính nhất quán (ACID)

Nếu xảy ra lỗi trong quá trình thực thi (ví dụ: lỗi khi tạo đơn hàng), toàn bộ transaction sẽ được rollback, đảm bảo dữ liệu không bị rơi vào trạng thái không nhất quán.

4.8.5 Giai đoạn phản hồi dữ liệu

Sau khi Database trả về kết quả, dữ liệu được chuyển ngược lên tầng Service. Tại đây, Service có thể thực hiện các xử lý bổ sung nếu cần (mapping, tổng hợp dữ liệu, loại bỏ thông tin không cần thiết).

Trước khi trả về client, dữ liệu được ánh xạ sang Response DTO, đảm bảo:

- Cấu trúc dữ liệu phản hồi nhất quán

- Các thông tin nhạy cảm (như mật khẩu, token nội bộ) không bị lộ ra ngoài
- Dữ liệu phù hợp với hợp đồng API đã công bố

Cuối cùng, Router trả response về cho client dưới dạng JSON kèm theo HTTP status code tương ứng.

4.9 Đánh giá kiến trúc và luồng hoạt động

Kiến trúc Router – Service – Repository cùng với luồng hoạt động được thiết kế mang lại các lợi ích quan trọng sau:

- **Testability (Khả năng kiểm thử):** Dễ dàng viết unit test cho từng tầng Service một cách độc lập
- **Maintainability (Khả năng bảo trì):** Thay đổi logic ở một tầng ít ảnh hưởng đến các tầng khác
- **Scalability (Khả năng mở rộng):** Dễ dàng mở rộng hệ thống với các module mới
- **Clean Code:** Mỗi layer có ranh giới và trách nhiệm rõ ràng, tuân thủ các nguyên tắc SOLID
- **Data Integrity (Toàn vẹn dữ liệu):** Dữ liệu được kiểm soát chặt chẽ từ đầu vào đến đầu ra
- **Security (Bảo mật):** Giảm thiểu lỗi nghiệp vụ và lỗi bảo mật thông qua validation pipeline

Kiến trúc này đặc biệt phù hợp với các hệ thống backend hiện đại, cụ thể là các ứng dụng thương mại điện tử có nhiều nghiệp vụ phức tạp và yêu cầu khả năng mở rộng trong dài hạn. Sự tách bạch rõ ràng giữa kiến trúc tĩnh (static structure) và luồng hoạt động động (dynamic behavior) không chỉ giúp team phát triển dễ dàng phân công công việc mà còn đảm bảo tính ổn định, linh hoạt và khả năng mở rộng cao của hệ thống trong tương lai.

5. CƠ CHẾ XÁC THỰC VÀ PHÂN QUYỀN

5.1 Tổng quan Authentication và Authorization

Hệ thống ShopApp phân tách rõ ràng hai quá trình bảo mật cốt lõi: **Xác thực (Authentication)** để định danh người dùng và **Phân quyền (Authorization)** để kiểm soát truy cập tài nguyên.

5.1.1 Authentication (Xác thực - Xác định “User là ai?”)

Hệ thống xây dựng cơ chế xác thực đa phương thức, kết quả cuối cùng đều trả về cặp **JWT (JSON Web Token)** để Client sử dụng cho các request tiếp theo.

- **Đăng nhập truyền thống (Local Account):**

- Hỗ trợ người dùng đăng nhập linh hoạt bằng cả **Số điện thoại** hoặc **Email** thông qua DTO đăng nhập thống nhất.
- Mật khẩu người dùng được bảo mật thông qua cơ chế hashing 2 lớp (SHA256 kết hợp Bcrypt), đảm bảo không lưu trữ mật khẩu gốc trong cơ sở dữ liệu.

- **Social Login (OAuth2):**

- Tích hợp xác thực thông qua bên thứ 3 là **Google** và **Facebook**.
- Server nhận Access Token từ Client, sau đó verify (xác thực) lại với Provider để lấy thông tin định danh (`sub`, `email`, `name`) thay vì xử lý mật khẩu trực tiếp.
- **Cơ chế đồng bộ (Just-in-Time Provisioning):** Nếu tài khoản Social chưa tồn tại, hệ thống tự động tạo User mới với mật khẩu ngẫu nhiên và gán quyền mặc định.

5.1.2 Authorization (Phân quyền - Xác định “User được làm gì?”)

Hệ thống áp dụng mô hình **RBAC (Role-Based Access Control)**, quản lý quyền hạn dựa trên vai trò được gán cho người dùng trong bảng `roles`.

- **Các vai trò (Roles):**

- **User (Khách hàng):** Quyền mặc định (`role_id = 1`). Có quyền thực hiện các tác vụ cá nhân như đặt hàng, xem lịch sử đơn hàng của chính mình.
- **Admin (Quản trị viên):** Có quyền truy cập toàn bộ tài nguyên hệ thống, bao gồm các chức năng nhạy cảm như quản lý danh mục, xóa dữ liệu và xem thống kê doanh thu.

- **Cơ chế kiểm soát:**

- Phân quyền được thực hiện tại tầng Service hoặc Router thông qua Dependency Injection.

- Sau khi xác thực qua `get_current_user`, hệ thống kiểm tra thuộc tính `role.name`. Nếu người dùng không phải Admin nhưng truy cập tài nguyên bị hạn chế, hệ thống trả về lỗi 403 Forbidden.

5.1.3 JWT Access/Refresh Token: Vòng đời, Luồng cấp mới và Thu hồi

Hệ thống sử dụng cơ chế xác thực **Dual Token** (Access Token kết hợp Refresh Token) và quản lý trạng thái token trong cơ sở dữ liệu để cân bằng giữa hiệu năng và bảo mật.

- **Vòng đời (Lifecycle):**

- **Access Token:** Có thời hạn ngắn (**30 phút**), được định dạng chuẩn JWT. Token này chứa các thông tin định danh (claims) như `sub`, `id`, `role_id`, giúp Server xác thực và phân quyền nhanh chóng mà không cần truy vấn DB liên tục.
- **Refresh Token:** Có thời hạn dài (**7 ngày**), được lưu trữ trong bảng `tokens` để quản lý trạng thái đăng nhập của thiết bị.

- **Luồng cấp mới (Token Rotation):** Hệ thống áp dụng kỹ thuật **Token Rotation** để tăng cường bảo mật. Khi Client gửi yêu cầu gia hạn token:

1. Kiểm tra Refresh Token trong database. Nếu token không tồn tại, đã hết hạn hoặc đã bị thu hồi thì từ chối yêu cầu.
2. Nếu hợp lệ, hệ thống tạo mới một Access Token và một Refresh Token.
3. Cập nhật đề token mới lên bản ghi cũ trong database nhằm ngăn chặn replay attack.

- **Cơ chế thu hồi (Revocation):** Trạng thái token được kiểm soát thông qua cột `revoked` trong bảng `tokens`.

- Khi người dùng đăng xuất hoặc bị cưỡng chế đăng xuất, hệ thống cập nhật `revoked = True`.
- Mọi yêu cầu gia hạn token sau đó đều bị từ chối ngay lập tức.

5.1.4 Hash mật khẩu bằng Bcrypt

Hệ thống không lưu mật khẩu dạng văn bản thuần mà áp dụng cơ chế **Double Hashing** để tăng cường bảo mật.

- **Quy trình Hash:**

1. Mật khẩu đầu vào được băm bằng SHA256 để chuẩn hóa độ dài dữ liệu.
2. Kết quả SHA256 tiếp tục được băm bằng Bcrypt với salt ngẫu nhiên.

- **Quy trình Verify:** Khi người dùng đăng nhập, mật khẩu gửi lên được băm SHA256 trước, sau đó so sánh với hash đã lưu bằng hàm `verify` của Bcrypt.

5.1.5 OAuth2 Social Login (Google/Facebook)

Hệ thống hỗ trợ đăng nhập nhanh thông qua Google và Facebook, giúp tăng trải nghiệm người dùng (UX) và tỷ lệ chuyển đổi.

- **Luồng đăng nhập và Tự động tạo User (Auto-provisioning):**

1. Client (Frontend) thực hiện đăng nhập với Provider và gửi `accessToken` (Facebook) hoặc `credential` (Google) lên Server.
2. **Verify Token:** Server không tin tưởng dữ liệu từ Client mà trực tiếp xác thực lại với Provider:
 - **Google:** Sử dụng thư viện `google-auth` để gọi hàm `verify_oauth2_token`, đảm bảo token được ký bởi Google và cấp cho đúng Client ID của ứng dụng.
 - **Facebook:** Gọi Graph API (<https://graph.facebook.com/me>) để lấy thông tin định danh mới nhất.
3. **Xử lý Logic User:**
 - Hệ thống tìm kiếm User trong DB dựa trên `facebook_account_id` hoặc `google_account_id`.
 - Nếu chưa tồn tại, hệ thống thực hiện **Just-in-Time Provisioning**: tự động tạo một User mới với mật khẩu ngẫu nhiên (UUID) và kích hoạt ngay lập tức.
 - Cuối cùng, hệ thống trả về cặp Access/Refresh Token của ShopApp để User sử dụng như tài khoản thường.

- **Rủi ro và Cách xử lý:**

- **Token giả mạo:** Việc backend tự gọi xác thực lại với Provider (Back-channel communication) giúp loại bỏ rủi ro Client gửi token giả hoặc token đã hết hạn.
- **Email Trust:** Với Google Login, hệ thống ưu tiên sử dụng email đã được verify từ Google ID Token để liên kết tài khoản, đảm bảo tính chính xác.

5.1.6 RBAC (Role-Based Access Control)

Hệ thống kiểm soát quyền truy cập dựa trên vai trò (Role) của người dùng, được thực hiện chặt chẽ từ tầng Authentication Middleware đến tầng Service Logic.

- **Dependency `get_current_user`:** Đây là "cánh cổng" đầu tiên bảo vệ các Router nhạy cảm. Hàm này thực hiện:
 1. Giải mã JWT Token từ Header Authorization.
 2. Truy vấn Database để lấy thông tin User đầy đủ kèm theo Role tương ứng.
 3. Nếu User không tồn tại hoặc bị khóa, lập tức trả về lỗi 401 `Unauthorized`.

- **Kiểm tra Role (Role Guard):** Tại các Router hoặc Service nhạy cảm, hệ thống thực hiện kiểm tra quyền Admin một cách tường minh:
 - Ví dụ trong `category_service.py`: Trước khi xóa danh mục, hệ thống kiểm tra `if current_user.role.name != 'admin'`.
 - Ví dụ trong `statistic_router.py`: `Dependency require_admin` được viết riêng để tái sử dụng logic chặn truy cập đối với các API thống kê doanh thu.
- **Phân quyền Admin/User:**
 - **User (Role ID 1):** Chỉ có quyền truy cập các tài nguyên công khai hoặc tài nguyên sở hữu riêng.
 - **Admin (Role ID 2):** Có quyền thực thi các hành động làm thay đổi hệ thống và xem các dữ liệu nhạy cảm như dashboard thống kê.

6. Frontend

6.1 Tổng quan về Front-end

Front-end là phần giao diện của hệ thống, cho phép người dùng trực tiếp tương tác với ứng dụng thông qua trình duyệt web. Nhiệm vụ chính của Front-end là hiển thị dữ liệu, tiếp nhận thông tin từ người dùng và gửi các yêu cầu tương ứng tới hệ thống Back-end.

Trong dự án xây dựng trang web bán hàng, Front-end đóng vai trò quan trọng trong việc đảm bảo trải nghiệm người dùng (UX), tính trực quan của giao diện (UI) cũng như khả năng sử dụng và tương tác hiệu quả với hệ thống.

6.1.1 React trong phát triển Front-end

React là một thư viện JavaScript mã nguồn mở được sử dụng rộng rãi để xây dựng giao diện người dùng theo mô hình component. Thay vì cập nhật toàn bộ trang khi dữ liệu thay đổi, React chỉ cập nhật những thành phần cần thiết thông qua cơ chế Virtual DOM, từ đó giúp cải thiện hiệu năng và tăng tính mượt mà cho ứng dụng.

Trong dự án, React được sử dụng để xây dựng các trang chính như trang chủ, trang danh sách sản phẩm, trang chi tiết sản phẩm, trang đăng nhập và trang đăng ký. Việc chia giao diện thành các component nhỏ giúp mã nguồn dễ quản lý, dễ mở rộng và tăng khả năng tái sử dụng trong quá trình phát triển.

6.1.2 Các React Hooks được sử dụng

Hook `useState` Hook `useState` được sử dụng để quản lý trạng thái cục bộ của component. Trong dự án, hook này được dùng để lưu trữ các giá trị nhập liệu của người dùng như email và mật khẩu, trạng thái hiển thị hoặc ẩn mật khẩu, cũng như các trạng thái kiểm tra lỗi trong biểu mẫu đăng nhập và đăng ký. Khi giá trị state thay đổi, giao diện sẽ được React tự động cập nhật tương ứng.

Hook `useEffect` Hook `useEffect` được sử dụng để xử lý các tác vụ phụ như gọi API, tải dữ liệu hoặc đồng bộ trạng thái của ứng dụng. Trong dự án, `useEffect` được sử dụng để gọi API lấy danh sách sản phẩm và danh mục khi trang được tải lần đầu, đồng thời đảm bảo dữ liệu được cập nhật khi trạng thái của ứng dụng thay đổi.

Hook `useNavigate` Hook `useNavigate` thuộc thư viện React Router DOM, được sử dụng để điều hướng giữa các trang trong ứng dụng mà không cần tải lại toàn bộ website. Trong dự án, hook này được sử dụng khi người dùng nhấn vào một sản phẩm để chuyển sang trang chi tiết, hoặc khi hoàn thành các thao tác như đăng nhập và đăng ký tài khoản thành công.

6.1.3 Material UI (MUI)

Material UI (MUI) là một thư viện giao diện người dùng dành cho React, được xây dựng dựa trên chuẩn thiết kế Material Design của Google. MUI cung cấp sẵn nhiều thành phần giao

diện giúp việc xây dựng UI trở nên nhanh chóng, đồng bộ và dễ dàng tùy chỉnh theo yêu cầu của dự án.

Trong dự án, Material UI được sử dụng để xây dựng toàn bộ giao diện Front-end, từ bố cục tổng thể đến các thành phần tương tác chi tiết với người dùng.

6.1.4 Các thành phần Material UI sử dụng trong dự án

Các component Material UI tiêu biểu được sử dụng bao gồm:

- **Box**: Thành phần bố cục tổng quát, dùng để sắp xếp layout, căn chỉnh khoảng cách và quản lý vị trí các phần tử giao diện.
- **Typography**: Thành phần hiển thị văn bản, giúp quản lý kiểu chữ, cỡ chữ và tiêu đề một cách nhất quán.
- **Button**: Thành phần nút bấm, được sử dụng cho các chức năng như đăng nhập, đăng ký và thêm sản phẩm vào giỏ hàng.
- **TextField**: Thành phần ô nhập liệu, dùng trong các form nhập thông tin người dùng.
- **Paper**: Thành phần tạo khung nền dạng thẻ, thường được sử dụng để bao bọc các form hoặc nội dung chính.
- **Divider**: Thành phần đường phân cách nội dung, giúp giao diện rõ ràng và dễ theo dõi hơn.
- **IconButton**: Nút bấm chứa biểu tượng, được sử dụng cho các thao tác nhanh như hiển thị hoặc ẩn mật khẩu.
- **Switch** và **FormControlLabel**: Thành phần tùy chọn dạng bật/tắt, dùng cho các tùy chọn cá nhân của người dùng.

6.1.5 Kết luận

Việc sử dụng React kết hợp với Material UI giúp xây dựng giao diện người dùng hiện đại, linh hoạt và dễ mở rộng. React đảm nhiệm việc quản lý dữ liệu, trạng thái và điều hướng của ứng dụng, trong khi Material UI cung cấp các thành phần giao diện nhất quán và thân thiện với người dùng. Sự kết hợp này góp phần nâng cao chất lượng tổng thể và tính chuyên nghiệp của hệ thống.

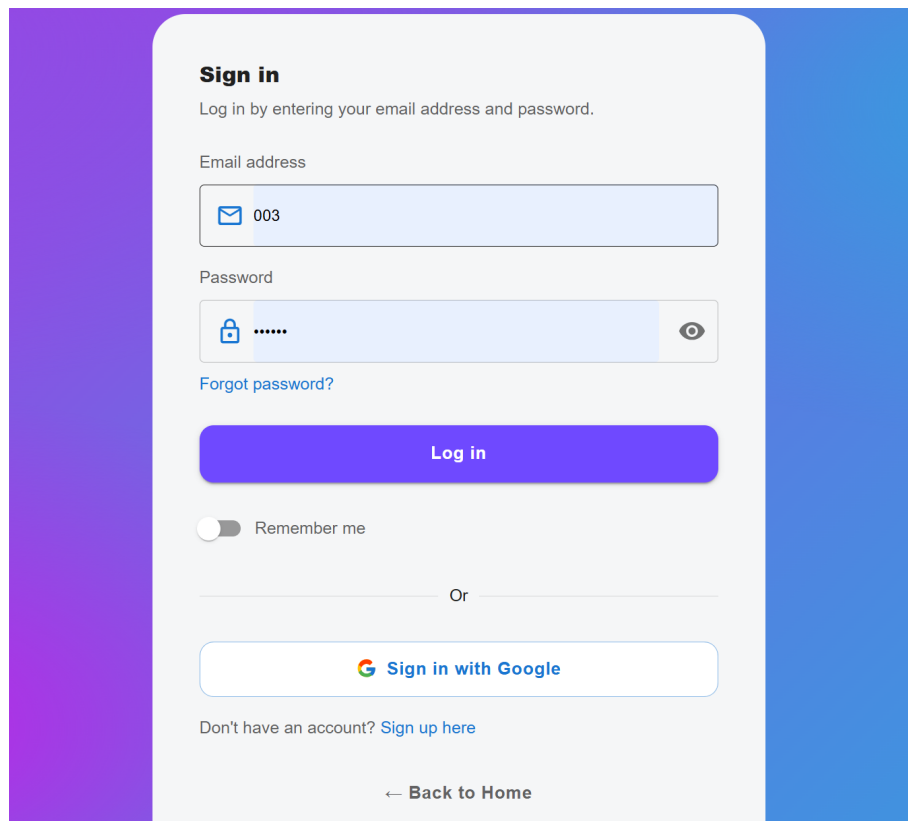
6.2 Thiết kế các trang xác thực tài khoản

6.2.1 Trang đăng nhập (Sign In)

Trang đăng nhập cho phép người dùng truy cập hệ thống bằng **email** và **mật khẩu**. Giao diện gồm các thành phần chính:

- **Email address:** ô nhập email người dùng.
- **Password:** ô nhập mật khẩu, có nút (**icon mắt**) để hiển thị/ẩn mật khẩu.
- **Log in:** gửi thông tin đăng nhập, nếu hợp lệ sẽ điều hướng về trang chủ.
- **Forgot password?:** chuyển sang trang quên mật khẩu.
- **Sign up here:** chuyển sang trang đăng ký tài khoản mới.
- **Back to Home:** quay về trang chủ.

Hình minh họa:



The image shows a 'Sign in' form on a light gray background with a purple and blue gradient border. The form is titled 'Sign in' and includes the instruction 'Log in by entering your email address and password.' It features two input fields: 'Email address' with a blue envelope icon and the text '003', and 'Password' with a blue lock icon, masked dots, and a blue eye icon for toggling visibility. Below the password field is a blue link 'Forgot password?'. A large blue button labeled 'Log in' is positioned below the fields. Underneath is a toggle switch for 'Remember me'. A horizontal line with the word 'Or' in the center separates this from a white button with a blue border labeled 'Sign in with Google'. At the bottom, there is a link 'Don't have an account? Sign up here' and a link '← Back to Home'.

Hình 2: Giao diện đăng nhập

6.2.2 Trang đăng ký (Sign Up)

Hình minh họa:

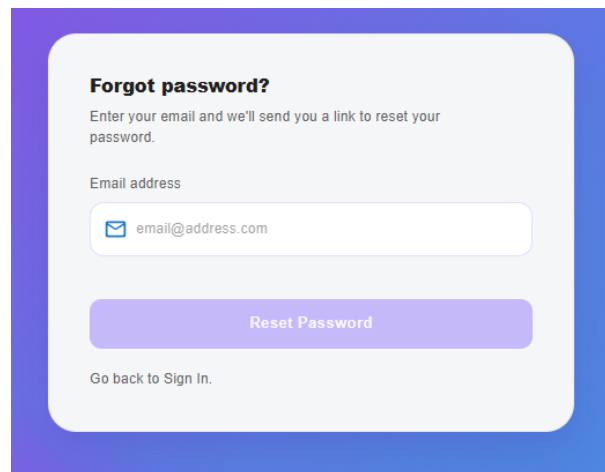
Hình 3: Giao diện đăng ký

Trang đăng ký cho phép người dùng tạo tài khoản mới bằng cách nhập đầy đủ thông tin cá nhân. Các thành phần chính:

- **Full name:** họ và tên người dùng.
- **Phone number:** số điện thoại.
- **Address:** địa chỉ.
- **Email address:** email đăng ký.
- **Password:** mật khẩu .
- **Confirm password:** nhập lại mật khẩu để xác nhận
- **Create Account:** tạo tài khoản (chỉ bật khi dữ liệu hợp lệ).
- **Log in here:** quay lại trang đăng nhập.
- **Back to Home:** quay về trang chủ.

6.2.3 Trang quên mật khẩu (Forgot Password)

Hình minh họa: (tự chèn hình giao diện quên mật khẩu)



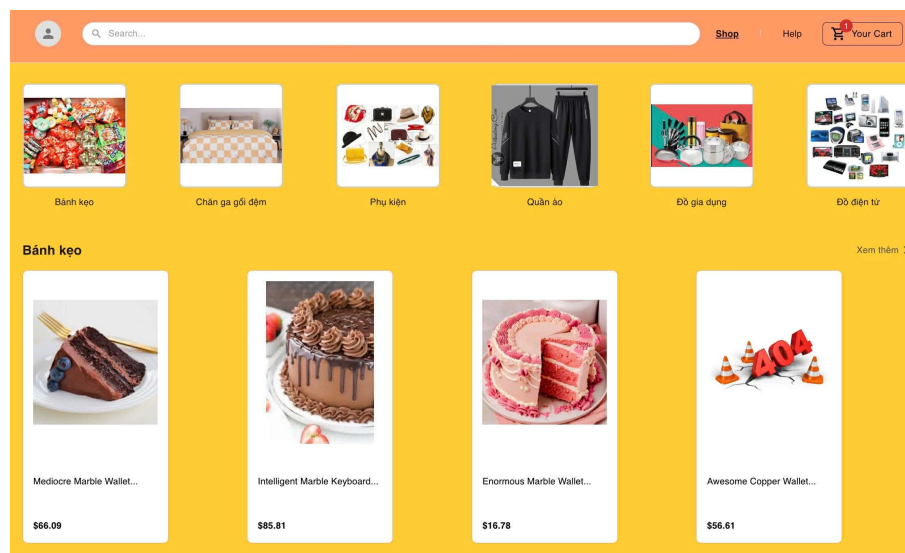
Hình 4: Giao diện quên mật khẩu

Trang quên mật khẩu cho phép người dùng nhập email để yêu cầu đặt lại mật khẩu. Các thành phần chính:

- **Email address:** nhập email đã đăng ký.
- **Reset Password:** gửi yêu cầu khôi phục mật khẩu.
- **Go back to Sign In:** quay lại trang đăng nhập.

6.2.4 Trang Home

Hình minh họa:



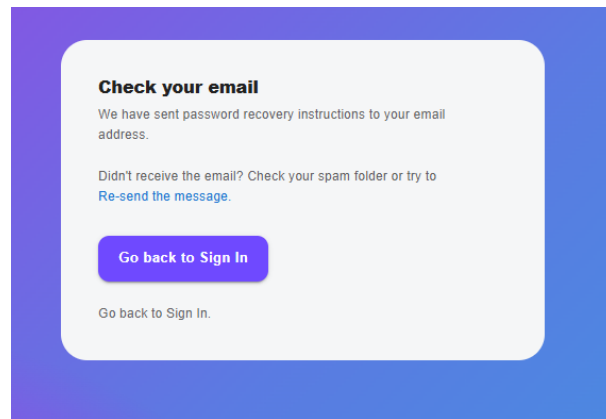
Hình 5: Trang HomeHome

Trang Home đóng vai trò là trang khởi đầu của hệ thống, giúp người dùng có cái nhìn tổng quan về website ngay từ lần truy cập đầu tiên. Trang này cung cấp các thông tin giới thiệu chung, định hướng người dùng đến các chức năng chính như xem danh sách sản phẩm, tìm kiếm hoặc truy cập vào các khu vực quan trọng khác trong hệ thống.

Giao diện trang Home được thiết kế đơn giản, trực quan nhằm tạo ấn tượng ban đầu tốt cho người dùng, đồng thời hỗ trợ việc điều hướng nhanh chóng và thuận tiện. Việc bố trí nội dung hợp lý trên trang Home góp phần nâng cao trải nghiệm người dùng, giúp người dùng dễ dàng tiếp cận và sử dụng các chức năng của hệ thống một cách hiệu quả.

6.2.5 Trang kiểm tra email (Check Email)

Hình minh họa:



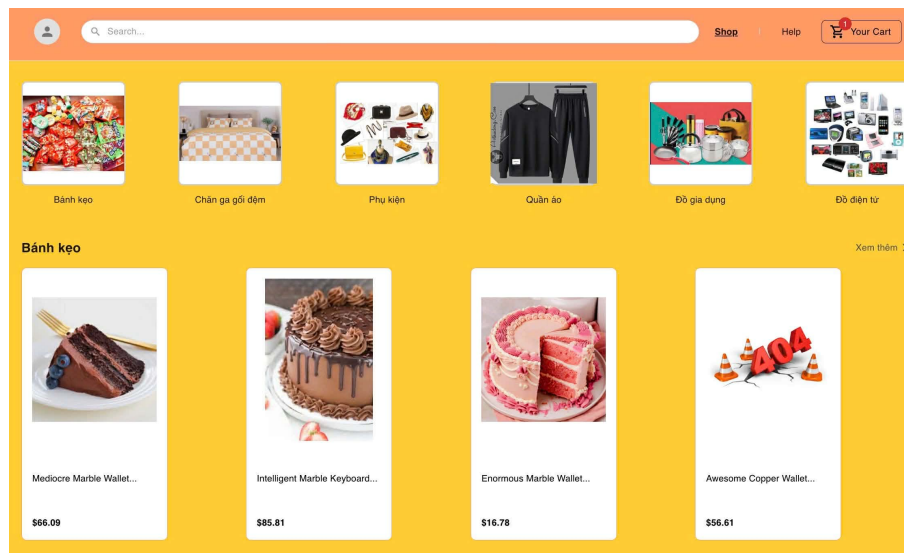
Hình 6: Giao diện kiểm tra email

Sau khi người dùng yêu cầu khôi phục mật khẩu, hệ thống hiển thị trang thông báo đã gửi email hướng dẫn. Các thành phần chính:

- **Thông báo:** xác nhận email hướng dẫn đã được gửi.
- **Re-send the message:** gửi lại email nếu người dùng chưa nhận được.
- **Go back to Sign In:** quay lại trang đăng nhập.

6.2.6 Trang Home

Hình minh họa:



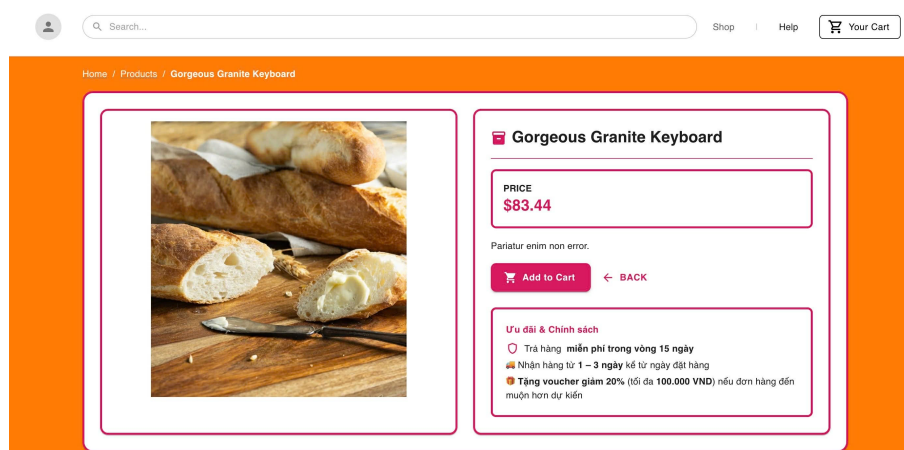
Hình 7: Trang HomeHome

Trang Home đóng vai trò là trang khởi đầu của hệ thống, giúp người dùng có cái nhìn tổng quan về website ngay từ lần truy cập đầu tiên. Trang này cung cấp các thông tin giới thiệu chung, định hướng người dùng đến các chức năng chính như xem danh sách sản phẩm, tìm kiếm hoặc truy cập vào các khu vực quan trọng khác trong hệ thống.

Giao diện trang Home được thiết kế đơn giản, trực quan nhằm tạo ấn tượng ban đầu tốt cho người dùng, đồng thời hỗ trợ việc điều hướng nhanh chóng và thuận tiện. Việc bố trí nội dung hợp lý trên trang Home góp phần nâng cao trải nghiệm người dùng, giúp người dùng dễ dàng tiếp cận và sử dụng các chức năng của hệ thống một cách hiệu quả.

6.2.7 Trang Product

Hình minh họa:



Hình 8: Trang product

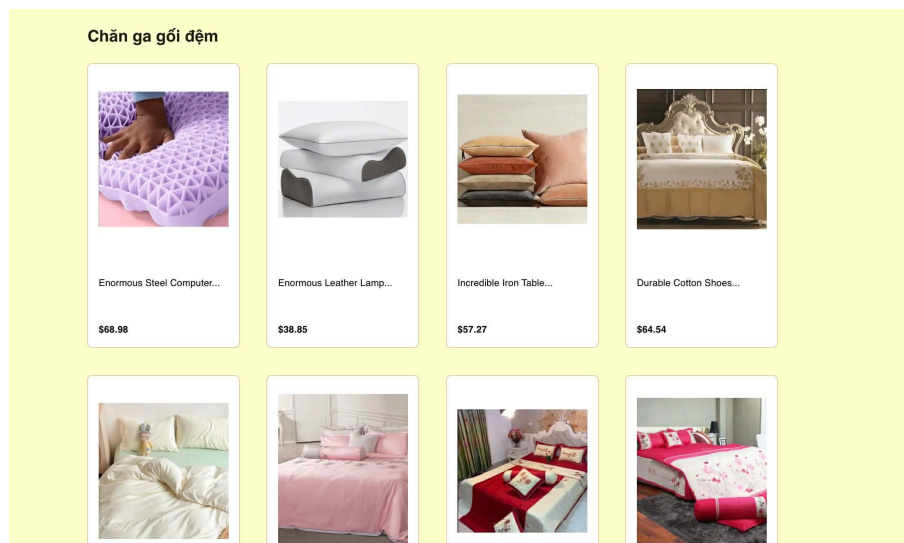
Trang sản phẩm hiển thị danh sách các sản phẩm hiện có trong hệ thống cùng các thông tin cơ bản như hình ảnh minh họa, tên sản phẩm và giá bán. Trang được xây dựng trên nền tảng React với mục tiêu cung cấp giao diện trực quan và dễ sử dụng cho người dùng.

Khi trang được tải, dữ liệu sản phẩm được lấy từ hệ thống backend và lưu trữ để phục vụ việc hiển thị. Trạng thái dữ liệu được quản lý tại phía Frontend nhằm đảm bảo giao diện có thể cập nhật linh hoạt theo dữ liệu nhận được.

Người dùng có thể tương tác với từng sản phẩm thông qua các chức năng như xem chi tiết thông tin hoặc thêm sản phẩm vào giỏ hàng. Các thao tác này giúp nâng cao trải nghiệm người dùng và hỗ trợ quá trình mua sắm trực tuyến một cách thuận tiện.

6.2.8 Trang danh mục sản phẩm

Hình minh họa:



Hình 9: Trang danh mục sản phẩm

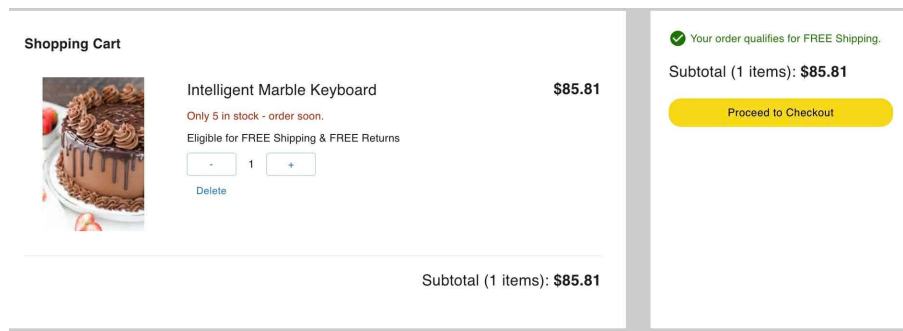
Trang danh mục sản phẩm được xây dựng nhằm hỗ trợ người dùng xem và tìm kiếm các sản phẩm theo từng danh mục cụ thể trong hệ thống. Việc phân loại sản phẩm theo danh mục giúp người dùng dễ dàng tiếp cận các sản phẩm phù hợp với nhu cầu, đồng thời nâng cao trải nghiệm sử dụng.

Khi người dùng lựa chọn một danh mục, hệ thống sẽ hiển thị danh sách các sản phẩm thuộc danh mục đó cùng các thông tin cơ bản như hình ảnh minh họa, tên sản phẩm và giá bán. Dữ liệu hiển thị được lấy từ hệ thống backend và được cập nhật tương ứng với danh mục được chọn.

Trang danh mục vẫn cung cấp các chức năng tương tác cơ bản như xem chi tiết sản phẩm hoặc thêm sản phẩm vào giỏ hàng. Giao diện được thiết kế đồng bộ với các trang khác trong hệ thống nhằm đảm bảo tính nhất quán, trực quan và thuận tiện cho người dùng.

6.2.9 Trang Shopping Cart

Hình minh họa:



Hình 10: Trang Shopping Cart

Trang Shopping Cart cho phép người dùng xem lại các sản phẩm đã được lựa chọn trong quá trình mua sắm. Tại đây, người dùng có thể theo dõi danh sách sản phẩm trong giỏ hàng cùng các thông tin chi tiết như hình ảnh, tên sản phẩm, giá bán và số lượng của từng sản phẩm.

Trang Shopping Cart hỗ trợ các chức năng tương tác như tăng hoặc giảm số lượng sản phẩm, cũng như xóa sản phẩm khỏi giỏ hàng nếu người dùng không còn nhu cầu mua. Bên cạnh đó, hệ thống tự động cập nhật giá tiền của từng sản phẩm và tính toán tổng số tiền cần thanh toán dựa trên các thay đổi mà người dùng thực hiện.

Nhờ đó, trang Shopping Cart giúp người dùng dễ dàng quản lý đơn hàng trước khi tiến hành thanh toán, góp phần nâng cao trải nghiệm mua sắm và đảm bảo tính minh bạch trong quá trình đặt hàng.

6.2.10 Trang Thanh Toán và Thông Tin Giao Hàng

Hình minh họa:

Hình 11: Trang thanh toán

Trang Thanh Toán và Thông Tin Giao Hàng là bước cuối cùng trong quy trình đặt hàng của hệ thống. Tại trang này, người dùng sẽ nhập các thông tin cá nhân cần thiết để hoàn tất đơn hàng, bao gồm email, họ tên, số điện thoại và địa chỉ giao hàng. Trong trường hợp địa chỉ giao hàng khác với địa chỉ mặc định, người dùng có thể chỉnh sửa hoặc nhập thông tin mới cho phù hợp.

Bên cạnh đó, trang thanh toán cho phép người dùng lựa chọn phương thức vận chuyển và thêm ghi chú nếu có các yêu cầu đặc biệt đối với đơn hàng. Thông tin đơn hàng, bao gồm danh sách sản phẩm đã chọn, số lượng và tổng số tiền cần thanh toán, được hiển thị ở khu vực bên cạnh nhằm giúp người dùng dễ dàng kiểm tra lại trước khi xác nhận đặt hàng.

Trang cũng cung cấp các nút chức năng cho phép người dùng tiến hành đặt hàng hoặc quay lại các trang trước đó như giỏ hàng hoặc trang sản phẩm trong trường hợp không còn nhu cầu mua sắm. Việc thiết kế trang thanh toán rõ ràng và đầy đủ giúp đảm bảo tính chính xác của đơn hàng cũng như nâng cao trải nghiệm người dùng trong quá trình mua sắm trực tuyến.

6.3 Triển khai cơ chế kết nối dữ liệu phía Frontend

Trong hệ thống, việc kết nối và quản lý dữ liệu phía Frontend được thiết kế theo hướng tách biệt giữa phần xử lý dữ liệu và phần hiển thị giao diện. Cách tiếp cận này giúp tăng tính tổ chức của mã nguồn, đồng thời hỗ trợ mở rộng và bảo trì hệ thống trong quá trình phát triển.

Dữ liệu được lấy từ phía backend thông qua các yêu cầu bất đồng bộ và được xử lý trước khi đưa vào sử dụng trong giao diện người dùng. Trong quá trình này, dữ liệu đầu vào được kiểm tra và chuẩn hóa nhằm đảm bảo phù hợp với cấu trúc hiển thị và tránh các lỗi phát sinh trong quá trình render giao diện.

Trạng thái dữ liệu phía Frontend được quản lý tập trung, bao gồm dữ liệu chính, trạng thái tải và thông tin lỗi. Việc quản lý trạng thái tập trung giúp giao diện có thể phản hồi chính xác trong các tình huống đang tải dữ liệu, tải thành công hoặc khi xảy ra lỗi từ phía hệ thống.

Hệ thống hỗ trợ truyền các tham số truy vấn như phân trang hoặc bộ lọc dữ liệu để tối ưu hiệu năng và trải nghiệm người dùng. Các tham số này được chuẩn hóa trước khi gửi đi nhằm đảm bảo tính tương thích với backend.

Toàn bộ vòng đời của quá trình lấy dữ liệu được kiểm soát một cách nhất quán, giúp giao diện hiển thị trạng thái chờ, nội dung dữ liệu hoặc thông báo lỗi một cách rõ ràng và ổn định.

Tài liệu tham khảo

- [1] Oracle Corporation. (2024). MySQL 8.0 Reference Manual. *MySQL Official Documentation*. <https://dev.mysql.com/doc/refman/8.0/en/>
- [2] FastAPI Contributors. (2024). FastAPI Documentation. *FastAPI Official Documentation*. <https://fastapi.tiangolo.com/>
- [3] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). *RFC 7519, Internet Engineering Task Force (IETF)*. <https://datatracker.ietf.org/doc/html/rfc7519>
- [4] Meta Platforms, Inc. (2024). AccessibilityInfo – React Native Documentation. *React Native Official Documentation*. <https://reactnative.dev/docs/accessibilityinfo>
- [5] MUI Team. (2024). Typography Customization – Material UI. *Material UI Documentation*. <https://mui.com/material-ui/customization/typography/>